Assembler directives of 8086

Assembler directives of 8086

- Instructions to the Assembler regarding the program being executed.
- Control the generation of machine codes and organization of the program; but no machine codes are generated for assembler directives. These are also called 'pseudo instructions'

Assembler directives are used to :
› specify the start and end of a program.
› attach value to variables.
› allocate storage locations to input/ output data.
› define start and end of segments, procedures, macros etc..

Assembler directives of 8086

- DB
- DW
- SEGMENT
- ENDS
- ASSUME
- ORG
- END
- EVEN
- EQU
- PROC
- FAR
- NEAR
- ENDP
- SHORT
- MACRO
- ENDM

- **1. ASSUME** : The ASSUME directive is used to inform the assembler the name of the logical segment it should use for a specified segment.
- Ex: ASSUME DS: DATA tells the assembler that for any program instruction which refers to the data segment ,it should use the logical segment called DATA.
- **2.DB -**Define byte. It is used to declare a byte variable or set aside one or more storage locations of type byte in memory.
- For example, CURRENT_VALUE DB 36H tells the assembler to reserve 1 byte of memory for a variable named CURRENT_ VALUE and to put the value 36 H in that memory location when the program is loaded into RAM .
- **3. DW -Define word.** It tells the assembler to define a variable of type word or to reserve storage locations of type word in memory.
- **4**. **DD(define double word**) :This directive is used to declare a variable of type double word or restore memory locations which can be accessed as type double word.
- **5.DQ (define quadword) :**This directive is used to tell the assembler to declare a variable 4 words in length or to reserve 4 words of storage in memory .

- **6.DT (define ten bytes):**It is used to inform the assembler to define a variable which is **10** bytes in length or to reserve 10 bytes of storage in memory.
- **7. EQU –Equate** It is used to give a name to some value or symbol**.** Every time the assembler finds the given name in the program, it will replace the name with the value or symbol we have equated with that name
- **8.ORG -Originate** : The ORG statement changes the starting offset address of the data.
- It allows to set the location counter to a desired value at any point in the program. For example the statement ORG 3000H tells the assembler to set the location counter to 3000H.
- **9 .PROC**- Procedure: It is used to identify the start of a procedure. Or subroutine.
- **10. END**- End program .This directive indicates the assembler that this is the end of the program module.The assembler ignores any statements after an END directive.
- **11**. **ENDP**- End procedure: It indicates the end of the procedure (subroutine) to the assembler.
- **12.ENDS**-End Segment: This directive is used with the name of the segment to indicate the end of that logical segment.
- Ex: CODE SEGMENT : Start of logical segment containing code

  CODE ENDS : End of the segment named CODE.

DB: Define Byte

- Define a byte type (8-bit) variable.
- Reserves specific amount of memory locations to each variable.
- Range : 00H – FFH for unsigned value; 00H – 7FH for positive value and 80H – FFH for negative value.
- General form : variable DB value/ values.

- Example: LIST DB 7FH, 42H, 35H
- Three consecutive memory locations are reserved for the variable LIST and each data specified in the instruction are stored as initial value in the reserved memory location.

DW: Define word

- Define Word.
- Define a word type (16-bit) variable.
- Reserves two consecutive memory locations to each variable.
- Range : 0000H – FFFFH for unsigned value; 0000H – 7FFFH for positive value and 8000H – FFFFH for negative value.
- General form : variable DW value/ values.

- Example:
  ALIST DW 6512H, 0F251H, 0CDE2H
  Six consecutive memory locations are reserved for the variable ALIST and each 16-bit data specified in the instruction is stored in two consecutive memory location.

Segment ends

- SEGMENT : Used to indicate the beginning of a code/ data/
  stack segment
  ENDS : Used to indicate the end of a code/ data/ stack segment
  General form
- Segnam SEGMENT
  …
  …
  …
  …
  …
  …
  Segnam ENDS

Program code
or
Data Defining Statements

User defined name of the segment

ASSUME

- Informs the assembler the name of the program/ data segment that should be used for a specific segment.

- General form:
  ASSUME segreg : segnam, .. , segreg : segnam

  Example

  ASSUME CS: ACODE, DS:ADATA

- Tells the compiler that the instructions of the program are stored in the segment ACODE and data are stored in the segment ADATA

- **ORG** (Origin) is used to assign the starting address (Effective address) for a program/ data segment

- **END** is used to terminate a program; statements after END will be ignored.

- **EVEN** : Informs the assembler to store program/ data segment starting from an even address.

- **EQU** (Equate) is used to attach a value to a variable

Examples:

- **ORG** 1000H Informs the assembler that the statements following ORG 1000H should be stored in memory starting with effective address 1000H.

- LOOP EQU 10FEH

  Value of variable LOOP is 10FEH

```
_SDATA SEGMENT
  ORG 1200H
  A DB 4CH
  EVEN
  B DW 1052H
_SDATA ENDS
```

In this data segment, effective address of memory location assigned to A will be 1200H and that of B will be 1202H and 1203H.

PROC: **Procedure**

- PROC Indicates the beginning of a procedure ENDP End of procedure FAR Intersegment call NEAR Intrasegment call.

General form:

```
procname PROC[NEAR/ FAR]
...
...
...
RET
procname ENDP
```

Program statements of the procedure

Last statement of the procedure

User defined name of the procedure

- Reserves one memory location for 8-bit signed displacement in jump instructions.
- JMP SHORT AHEAD The directive will reserve one memory location for 8-bit displacement named AHEAD

Macro

- MACRO Indicate the beginning of a macro
  ENDM End of a macro
  General form:
  macroname MACRO[Arg1, Arg2 ...]
  macroname MACRO[Arg1, Arg2 ...]
  ...
  ...
  macroname ENDM

Program statements in the macro

User defined name of the macro